

LAPORAN PRAKTIKUM

STRUKTUR DATA

“Breadth First Search & Depth First
Search ”

disusun oleh: Dennis Shauqi Akbar

2511533020

Dosen pengampu: Dr. Wahyudi. S.T.M.T

Asisten Pratikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN

2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman. Tugas Individu ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, dengan Judul “BFS & DFS”

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pengampu yang telah memberikan arahan sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami.

Padang, 12 Juni 2026

Dennis Shauqi Akbar

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Pengertian Algoritma Sorting	2
2.2 Implementasi Pada Java.....	2
2.2.1 Node_2511533020.....	2
2.2.2 GraphTraversal_2511533020.....	3
2.2.3 Btree_251533020.....	5
2.2.4 QuickSort_2511533020	6
BAB III PENUTUP	8
3.1 Kesimpulan	8
3.2 Saran	8
3.3 Penutup	8
DAFTAR PUSTAKA.....	iii

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada struktur data graf dan pohon, proses untuk mengunjungi setiap simpul atau node dikenal dengan istilah traversal atau penelusuran. Dua metode yang umum digunakan adalah BFS (Breadth First Search) dan DFS (Depth First Search). BFS melakukan penelusuran secara bertahap berdasarkan level atau jarak dari simpul awal, sedangkan DFS lebih cenderung langsung masuk ke simpul terdalam sebelum kembali ke simpul sebelumnya. Kedua metode ini memiliki perbedaan prinsip baik dari segi implementasi maupun hasil urutan kunjungan.

Praktikum kali ini bertujuan untuk mengimplementasikan kedua algoritma tersebut dalam bahasa Java, baik pada graf maupun pada pohon biner. Selain itu, disediakan juga program untuk menampilkan bentuk pohon dan tiga jenis traversal DFS yaitu preorder, inorder, dan postorder.

1.2 Tujuan

Tujuan dari praktikum ini adalah memahami perbedaan konsep antara BFS dan DFS. Mahasiswa diharapkan mampu mengimplementasikan BFS menggunakan antrian (queue) dan DFS menggunakan rekursi. Selain itu, pada pohon biner kita juga belajar tentang tiga macam urutan traversal DFS beserta cara menampilkan struktur pohon.

1.3 Manfaat

Dengan mempelajari BFS dan DFS, mahasiswa dapat memilih algoritma traversal yang tepat sesuai dengan kasus yang dihadapi. BFS cocok untuk mencari jarak terpendek pada graf tak berbobot, sedangkan DFS berguna untuk eksplorasi data yang bersifat hierarkis atau untuk deteksi siklus.

Program yang dibuat pada praktikum ini juga dapat dikembangkan untuk keperluan lain seperti pencarian rute atau simulasi pohon keputusan.

BAB II

PEMBAHASAN

2.1 Pengertian Algoritma Sorting

BFS (Breadth First Search) adalah algoritma traversal yang mengunjungi semua simpul pada suatu level terlebih dahulu sebelum pindah ke level berikutnya. Implementasi BFS biasanya menggunakan struktur data antrian (queue) sehingga simpul yang pertama masuk akan pertama kali diproses (FIFO). Sedangkan DFS (Depth First Search) bekerja dengan mengunjungi simpul sedalam mungkin terlebih dahulu. Jika tidak ada lagi tetangga yang belum dikunjungi, maka proses mundur (backtrack). Implementasi DFS dapat dilakukan dengan rekursi atau dengan menggunakan stack.

Pada pohon biner, DFS menghasilkan tiga varian traversal yaitu preorder (kunjugi akar, kiri, kanan), inorder (kiri, akar, kanan), dan postorder (kiri, kanan, akar).

2.2 Implementasi Pada Java

Berikut adalah penjelasan untuk masing-masing kode program.

2.2.1 Node_2511533020

```
1 package pekan9_2511533020;
2
3 public class Node_2511533020 {
4     int data_3020;
5     Node_2511533020 left_3020;
6     Node_2511533020 right_3020;
7     public Node_2511533020(int data_3020) {
8         this.data_3020 = data_3020;
9         left_3020 = null;
10        right_3020 = null;
11    }
12    public void setLeft_3020(Node_2511533020 node) {
13        if (left_3020 == null)
14            left_3020 = node;
15    }
16    public void setRight_3020(Node_2511533020 node) {
17        if (right_3020 == null)
18            right_3020 = node;
19    }
20    public Node_2511533020 getLeft() {
21        return left_3020;
22    }
23    public Node_2511533020 getRight() {
24        return right_3020;
25    }
26    public int getData_3020() {
27        return data_3020;
28    }
29    public void setData(int data_3020) {
30        this.data_3020 = data_3020;
31    }
32
33    void printPreorder_3020(Node_2511533020 node) {
34        if (node == null)
35            return;
36        System.out.print(node.data_3020 + " ");
37        printPreorder_3020(node.left_3020);
38        printPreorder_3020(node.right_3020);
39    }
40    void printPostorder_3020(Node_2511533020 node) {
41        if (node == null)
42            return;
43        printPostorder_3020(node.left_3020);
44        printPostorder_3020(node.right_3020);
```

```

45     System.out.print(node.data_3020 + " ");
46 }
47 void printInOrder_3020(Node_2511533020 node) {
48     if (node == null)
49         return;
50     printInOrder_3020(node.left_3020);
51     System.out.print(node.data_3020 + " ");
52     printInOrder_3020(node.right_3020);
53 }
54 public String print_3020() {
55     return this.print("", true, "");
56 }
57 public String print(String prefix, boolean isTail, String sb) {
58     if (right_3020 != null) {
59         right_3020.print(prefix + (isTail ? "| " : " "), false, sb);
60     }
61     System.out.println(prefix + (isTail ? "\\-- " : "/-- ") + data_3020);
62     if (left_3020 != null) {
63         left_3020.print(prefix + (isTail ? " " : "| "), true, sb);
64     }
65     return sb;
66 }
67 }
68 }
69 }
70 }

```

(Gambar 2.2.1.1)

Kelas `Node_2511533020` digunakan untuk membuat simpul pohon biner. Ada tiga atribut yaitu `data_3020` (int), `left_3020`, dan `right_3020` yang merupakan referensi ke node kiri dan kanan. Konstruktor akan mengisi nilai `data_3020` dan mengatur `left_3020` serta `right_3020` menjadi null. Method `setLeft_3020` dan `setRight_3020` berfungsi untuk menambahkan node anak, tetapi hanya jika node anak tersebut belum ada. Untuk keperluan akses data, disediakan getter dan setter.

Kelas ini juga sudah mencakup method untuk traversal preorder, inorder, postorder secara rekursif. Yang menarik, ada method `print_3020` yang menampilkan struktur pohon secara visual dengan indentasi sehingga memudahkan pengecekan bentuk pohon.

2.2.2 GraphTraversal_2511533020

```

1 package pekan9_2511533020;
2
3 import java.util.*;
4 public class GraphTraversal_2511533020 {
5     private Map<String, List<String>> graph = new HashMap<>();
6
7     // Menambahkan edge (jika tak terakut)
8     public void addEdge_2511533020(String node1_3020, String node2_3020) {
9         graph.putIfAbsent(node1_3020, new ArrayList<>());
10        graph.putIfAbsent(node2_3020, new ArrayList<>());
11        graph.get(node1_3020).add(node2_3020);
12        graph.get(node2_3020).add(node1_3020);
13    }
14
15    // Menampilkan graf awal
16    public void printGraf_2511533020() {
17        System.out.println("Graf Awal (Adjacency List) : ");
18        for (String node_3020 : graph.keySet()) {
19            System.out.print(node_3020 + " > ");
20            List<String> neighbors_3020 = graph.get(node_3020);
21            System.out.println(String.join(", ", neighbors_3020));
22        }
23        System.out.println();
24    }
25
26    // DFS rekursif
27    public void dfs_2511533020(String start_3020) {
28        Set<String> visited_3020 = new HashSet<>();
29        System.out.println("Penelusuran DFS : ");
30        dfsHelper_2511533020(start_3020, visited_3020);
31        System.out.println();
32    }
33
34    private void dfsHelper_2511533020(String current_3020, Set<String> visited_3020) {
35        if (visited_3020.contains(current_3020)) return;
36        visited_3020.add(current_3020);
37        System.out.print(current_3020 + " ");
38        for (String neighbor_3020 : graph.getOrDefault(current_3020, new ArrayList<>())) {
39            dfsHelper_2511533020(neighbor_3020, visited_3020);
40        }
41    }
42
43    // BFS iteratif
44    public void bfs_2511533020(String start_3020) {
45        Set<String> visited_3020 = new HashSet<>();
46        Queue<String> queue_3020 = new LinkedList<>();
47        queue_3020.add(start_3020);
48    }
49 }

```

```

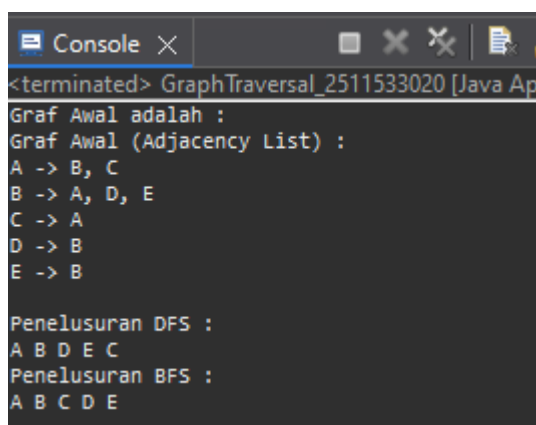
45     visited_3020.add(start_3020);
46     System.out.println("Penelusuran BFS : ");
47     while(!queue_3020.isEmpty()) {
48         String current_3020 = queue_3020.poll();
49         System.out.print(current_3020 + " ");
50         for(String neighbor : graph.getDefault(current_3020, new ArrayList<>()))
51             if(!visited_3020.contains(neighbor)) {
52                 queue_3020.add(neighbor);
53                 visited_3020.add(neighbor);
54             }
55     }
56     System.out.println();
57 }
58 //Main
59 public static void main(String[] args) {
60     GraphTraversal_2511533020 graph_3020 = new GraphTraversal_2511533020();
61
62     // contoh graf : A-B, A-C, B-D, B-E
63     graph_3020.addEdge_2511533020("A", "B");
64     graph_3020.addEdge_2511533020("A", "C");
65     graph_3020.addEdge_2511533020("B", "D");
66     graph_3020.addEdge_2511533020("B", "E");
67
68     // cetak graf awal
69     System.out.println("Graf Awal adalah : ");
70     graph_3020.printGraph_2511533020();
71     // lakukan penelusuran DFS
72     graph_3020.dfs_2511533020("A");
73     // lakukan penelusuran BFS
74     graph_3020.bfs_2511533020("A");
75 }

```

(Gambar 2.2.2.1)

Kelas ini khusus untuk BFS dan DFS pada graf. Representasi graf menggunakan adjacency list yaitu Map<String, List<String>> dengan nama variabel graph. Method addEdge_2511533020 digunakan untuk menambahkan sisi (edge) tak berarah. Pada method main, dibuat graf contoh dengan simpul A, B, C, D, E dan sisi A-B, A-C, B-D, B-E. Sebelum traversal, graf dicetak terlebih dahulu dalam bentuk daftar ketetanggaan. Method dfs_2511533020 melakukan penelusuran DFS dengan bantuan method rekursif dfsHelper_2511533020. Set visited digunakan untuk mencegah pengulangan simpul. Sedangkan bfs_2511533020 mengimplementasikan BFS dengan antrian (LinkedList) dan set visited. Hasil dari kedua metode dicetak ke layar.

Dari hasil percobaan, DFS dari simpul A menghasilkan urutan A, B, D, E, C, sedangkan BFS menghasilkan A, B, C, D, E. Perbedaan ini sesuai dengan prinsip kerja masing-masing algoritma.



```

<terminated> GraphTraversal_2511533020 [Java Ap
Graf Awal adalah :
Graf Awal (Adjacency List) :
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS :
A B D E C
Penelusuran BFS :
A B C D E

```

(Gambar 2.2.2.2)

2.2.3 Btree_251533020

```

1 package pekan9_2511533020;
2
3 public class Btree_2511533020 {
4     private Node_2511533020 root_3020;
5     private Node_2511533020 currentNode;
6     public Btree_2511533020() {
7         root_3020 = null;
8     }
9     public boolean search_3020(int data_3020) {
10         return search_3020(root_3020, data_3020);
11     }
12     private boolean search_3020(Node_2511533020 node, int data_3020) {
13         if (node.getData_3020() == data_3020)
14             return true;
15         if (node.getLeft() != null)
16             if (search_3020(node.getLeft(), data_3020))
17                 return true;
18         if (node.getRight() != null)
19             if (search_3020(node.getRight(), data_3020))
20                 return true;
21         return false;
22     }
23     public void printInorder_3020() {
24         root_3020.printInorder_3020 (root_3020);
25     }
26     public void printPreorder_3020() {
27         root_3020.printPreorder_3020(root_3020);
28     }
29     public void printPostorder_3020() {
30         root_3020.printPostorder_3020(root_3020);
31     }
32
33     public Node_2511533020 getRoot_3020() {
34         return root_3020;
35     }
36     public boolean isEmpty_3020() {
37         return root_3020 == null;
38     }
39
40     public int countNodes_3020() {
41         return countNodes_3020(root_3020);
42     }
43     private int countNodes_3020(Node_2511533020 node) {

```

```

45         int count_3020 = 1;
46         if (node == null) {
47             return 0;
48         } else {
49             count_3020 += countNodes_3020(node.getLeft());
50             count_3020 += countNodes_3020(node.getRight());
51             return count_3020;
52         }
53     }
54     public void print_3020() {
55         root_3020.print_3020();
56     }
57
58     public Node_2511533020 getCurrent_3020() {
59         return currentNode;
60     }
61
62     public void setCurrent_3020(Node_2511533020 node) {
63         this.currentNode = node;
64     }
65
66     public void setRoot_3020(Node_2511533020 root_3020) {
67         this.root_3020 = root_3020;
68     }
69     public void printPorstOrder_2511533020() {
70         // TODO Auto-generated method stub
71     }
72
73 }

```

(Gambar 2.2.3.1)

Kelas Btree_2511533020 merepresentasikan pohon biner secara keseluruhan. Atributnya ada root_3020 (simpul akar) dan currentNode (simpul yang sedang aktif). Konstruktor mengatur root menjadi null. Method search_3020 digunakan untuk mencari nilai tertentu di dalam pohon dengan cara rekursif DFS (preorder). Untuk keperluan traversal, disediakan method printInorder_3020, printPreorder_3020, printPostorder_3020 yang masing-masing memanggil

bentuk pohon menggunakan indentasi. Hasil traversal menunjukkan urutan yang sesuai dengan logika rekursif masing-masing metode.

```

<terminated> BtreeDriver_2511533020 [Java Application] C:\Users\d
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
Menampilkan simpul terakhir :
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder :
1 2 4 8 5 3 6 9 7
Postorden :
8 4 5 2 9 6 7 3 1
menampilkan simpul dalam bentuk pohon
|
|--7
|
|--3
|
|
|--6
|
|--9
|--1
|
|--5
|
|--2
|
|--8
|--4
  
```

(Gambar 2.2.4.2)

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, BFS dan DFS adalah dua algoritma traversal yang memiliki karakteristik berbeda. BFS menggunakan antrian dan menghasilkan urutan level, sedangkan DFS menggunakan rekursi atau stack dengan urutan mendalam. Implementasi pada graf menggunakan adjacency list dan pada pohon menggunakan node dengan pointer kiri dan kanan.

Ketiga traversal pada pohon (preorder, inorder, postorder) merupakan contoh penerapan DFS.

3.2 Saran

Dari hasil praktikum, ada beberapa hal yang bisa diperbaiki untuk pengembangan selanjutnya. Pada kelas `GraphTraversal_2511533020`, program saat ini hanya bisa menangani graf yang terhubung. Sebaiknya ditambahkan penanganan untuk graf yang tidak terhubung (disconnected graph) agar semua simpul tetap bisa dikunjungi. Untuk kelas `Btree_2511533020`, method `printPorstOrder_2511533020()` sebaiknya dihapus saja karena tidak digunakan dan menimbulkan kebingungan.

Pada metode pencarian (`search_3020()`), seharusnya ada pengecekan apakah node kosong terlebih dahulu sebelum mengakses `getData()`, karena jika node null akan terjadi error. Selain itu, untuk memudahkan pemahaman, praktikum berikutnya dapat menambahkan visualisasi langkah per langkah (misalnya dengan menunda eksekusi) agar alur BFS dan DFS lebih mudah diamati.

3.3 Penutup

Demikian laporan praktikum BFS dan DFS ini disusun. Laporan ini menjelaskan tentang konsep dasar kedua algoritma serta implementasinya dalam Java.

Penulis menyadari bahwa laporan ini belum sempurna, oleh karena itu kritik dan saran dari asisten sangat diharapkan

DAFTAR PUSTAKA

- [1] Oracle, "Queue (Java Platform SE 17)," Oracle. [Online]. Available: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Queue.html> (diakses: 11 Juni 2026).
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [3] Wikipedia, "Breadth first search," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Breadth-first_search (diakses: 11 Juni 2026).
- [4] Wikipedia, "Depth first search," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Depth-first_search (diakses: 11 Juni 2026).